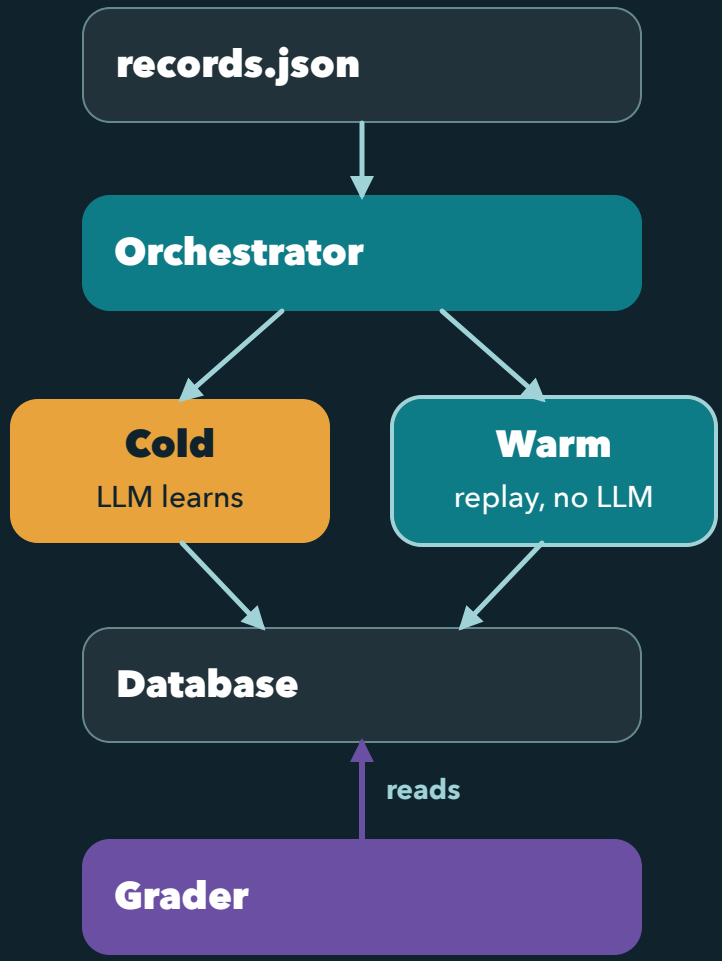
terminautomator v2

How the cache-aware pipeline is built, file by file

19
Python files

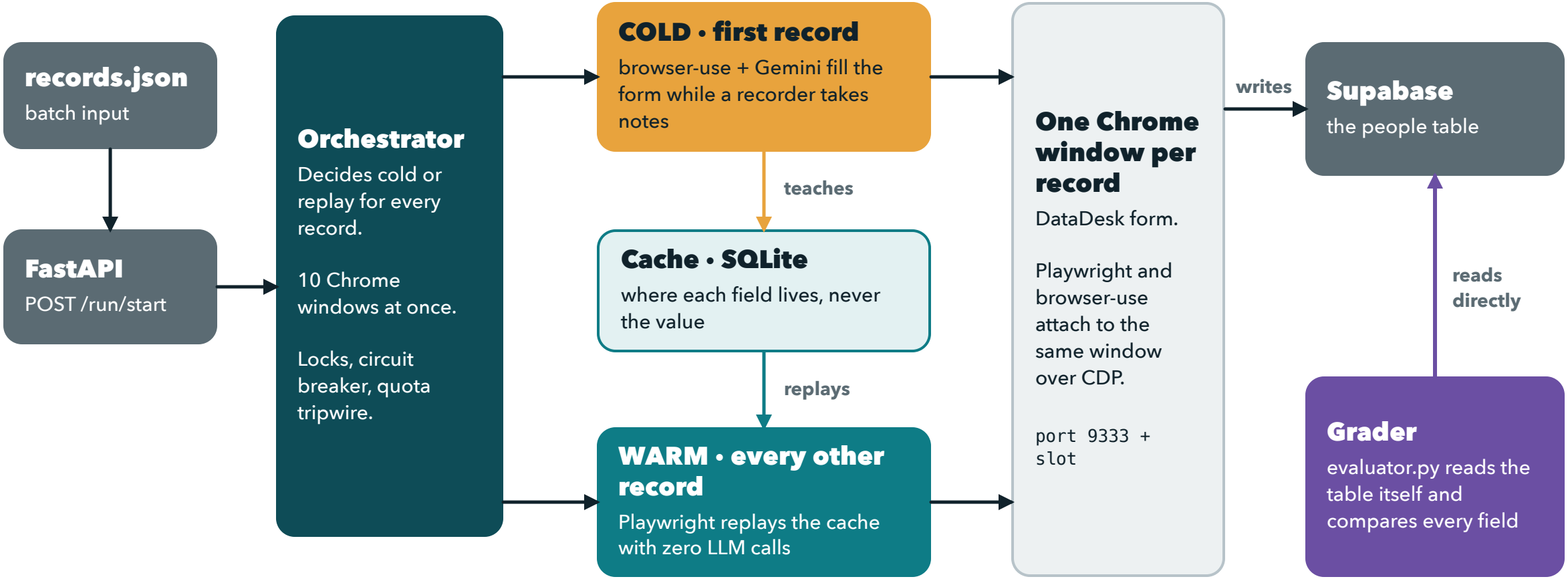
~2,900
lines of code

2
execution paths

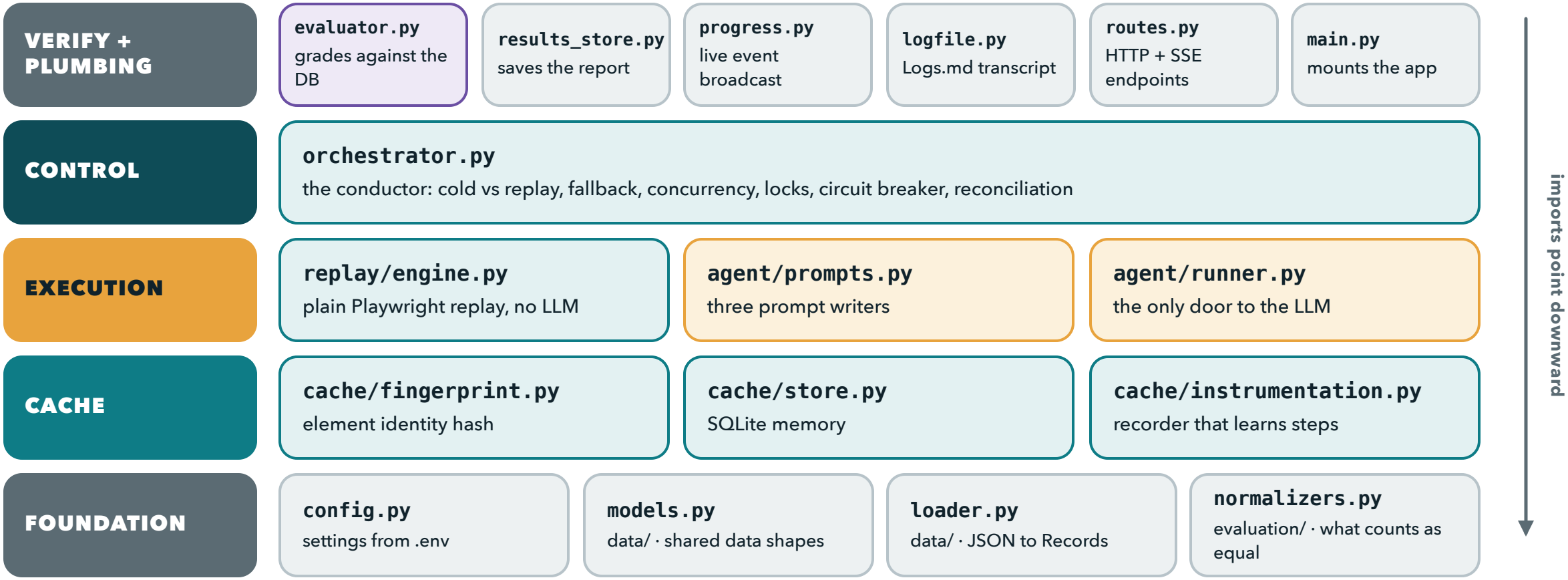


One pipeline, two speeds

uses the LLM · no LLM · independent



Five layers, one direction of dependency



The cache remembers where, never what

ONE ROW OF cached_steps (illustrative values)

workflow_name
datadesk_people_form

path_variant
default

target_key
department ★

action_type
select_dropdown

field_name
department

x_path
//form/div[3]/select

attributes
{"id":"field-department","name":"department"}

fingerprint
9f2c…e41a (sha-256)

instruction
agent's own goal text (diagnostic)

1 Identity is the field name

target_key is the field, or click::xpath for a click with no field. Relearning is an upsert: a newer attempt overwrites the older one, so rows never pile up.

2 No typed value is stored

The row says where the department field lives. On replay, each record types its own department into that spot.

3 One file, one lock

storage/cache.sqlite3 survives restarts. A thread lock wraps every call, because the batch thread and the reconciliation thread share one connection.

run_metrics table: mode · hits · misses · LLM steps per record → /cache/stats

Learning: a recorder watches the agent

only cache file coupled to browser-use

1

Hook

browser-use calls StepRecorder.callback before each step's actions run.

2

Find the element

Each action is mapped to the page element it targets, using the step's selector map.

3

Filter

Keep only click, input and select_dropdown. Waits, scrolls and done are ignored.

4

Which field?

Reverse-match the typed text to a record field with the normalizers. Checkboxes match by name or id.

5

Save

Build a CachedStep (xpath, attributes, fingerprint) and call store.upsert_step.

auto_persist on • cold run

Steps go straight into SQLite as they are learned.

auto_persist off • single-step fallback

The fallback agent restarts its step count at 1, so the orchestrator relabels the captured step with the original key, then saves it.

A claimed-field set

If two fields share a value (first and last name both John), each action claims a different field instead of overwriting one row.

Replay: find it, double-check it, do it

no LLM · plain Playwright

Tier 1 • exact XPath
Does the saved path match exactly one element?

exactly one

none or many

Tier 2 • attribute selector
data-testid › id › name › aria-label, first one the element had. Exactly one match?

exactly one

none or many

MISS → live fallback
reasons: no_match · execution_failed

Fingerprint check
Recompute the hash from the element found. A mismatch only logs a warning.

Where the value comes from
The orchestrator passes fields[step.field_name] from the current record. Plain clicks get no value.

perform(step, value)
Scroll into view, then one action by type. The value comes from the current record.

click
.click()

input
.fill(value)

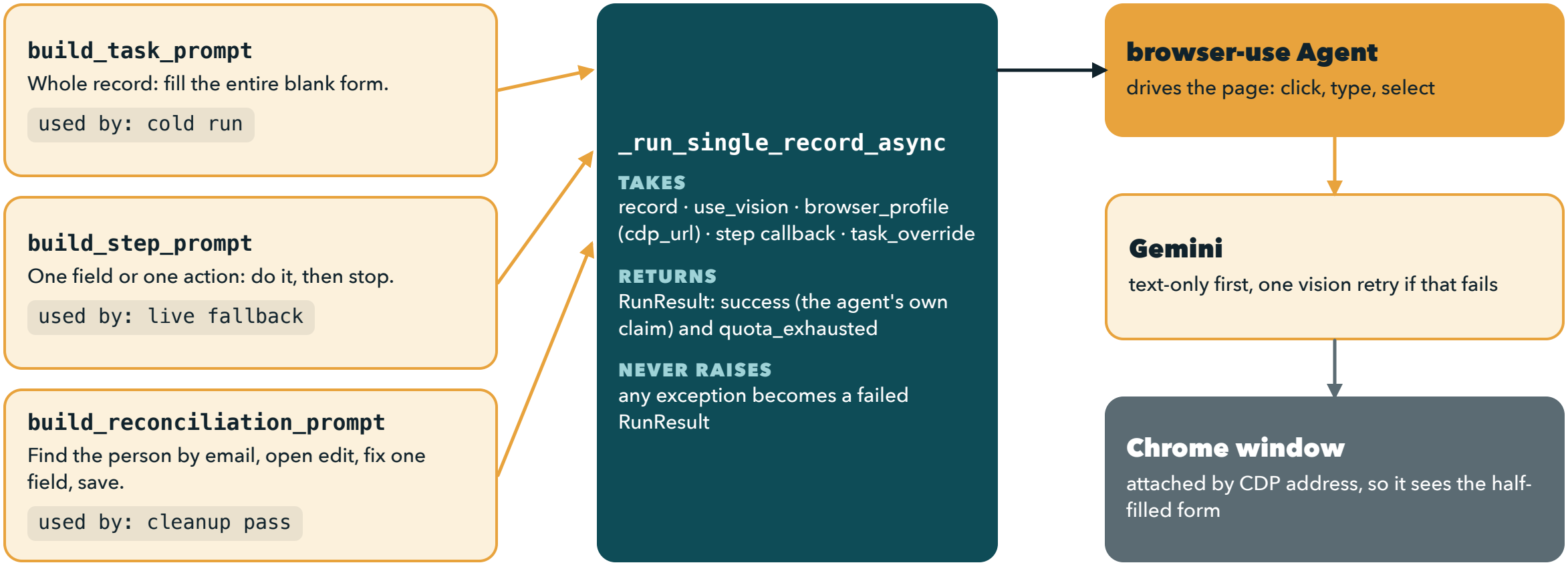
dropdown
.select_option(label)

checkbox
read state, click only if it differs

HIT

error → MISS

One door to the LLM



The orchestrator: the conductor

The other files each do one small job: the AI, the note-taker, the replayer, the cache and the grader.
The orchestrator decides **who does what, when, and what to do when something breaks.** It is the biggest file (~1000 lines) because it holds all the decisions.

THE THREE WAYS A RECORD CAN RUN

Cold

The cache is empty, so the AI fills the whole form and the note-taker learns it.

`_run_cold`

Replay

The cache exists, so the robot fills the form with no AI.

`_run_replay`

Live fallback

One replay step failed, so the AI fixes just that field, and the fix is saved.

`_run_live_fallback_step`

How many run at once

`_run_many_concurrent_async` runs up to 10 records at the same time, using a pool of numbered slots. Each slot has its own Chrome window and port (9333 + slot). When a record finishes, its slot is freed for the next waiting record.

It doesn't click, type or think, and it doesn't grade. It hands each job to the right helper.

One record, start to finish

1 Claim the seed
Cache empty and you win the lock: cold run. Otherwise replay.

→

2 Private Chrome
Temp profile, launched by Playwright on port 9333 + slot.

→

3 Open the form
Navigate to the form URL. Agent and replay share this tab.

→

4 Run it
_run_cold or _run_replay, inside a wait for the DB write.

→

5 Confirm the write
Wait for the POST to /rest/v1/people, 30 s. A timeout is only a warning.

→

6 Clean up
Close Chrome, release the lock, save run_metrics, emit record_done.

Why wait for the response

The submit click returns before the insert finishes. Closing the window too early cuts the write off, so the code waits for the real network response instead of a fixed sleep.

Failure stays local

The whole body is wrapped. Any exception becomes a failed RunResult for this record only, and a finally block always closes Chrome and releases the seed lock.

A miss: scoped fallback, then self-heal

A step can't replay
Replay returns no_match or execution_failed.

→

Circuit open?
This field failed twice in a row recently? Skip it instantly: no LLM, no wait.

→

One-field live agent
build_step_prompt: set only this field, then stop. Same Chrome tab, over CDP.

→

Self-heal
The recorder's capture is relabeled to the original step key and saved. The next record hits.

Fields the cache has never seen
The record has a value but the cache has no step for it (the seed record left notes empty, say). Only here does the per-field lock apply: one record asks the AI, the rest reuse the answer. The fix is saved just before the submit step.

What counts as a successful record
Only the submit step. One wrong optional field doesn't fail the record: the grader and reconciliation deal with it. Submit is the one step the circuit breaker never skips.

Six safety devices, one job each

1 Seed lock

Only one record cold-runs. The others wait until the cache is ready.

`_claim_seed_responsibility`

2 Per-field lock

If several records need the AI for the same missing field, only one asks, and the rest reuse the answer.

`_get_missing_field_lock`

3 Circuit breaker

If a field fails twice in a row, skip the AI for it for 30 seconds. It recovers by itself.

`_circuit_is_open`

4 Quota tripwire

If Gemini's daily limit is hit, stop everything. Retrying is pointless.

`_quota_exhausted_event`

5 Exception containment

One crashing record never takes down the batch.

`_run_one · asyncio.gather`

6 Response wait

After submit, it waits up to 30 seconds for the app's save request. A timeout is only a warning.

`page.expect_response`

The rest of its jobs

1 Batch entry point

What the API calls. It resets the locks, announces batch start, runs everything, and announces batch done.

`run_batch_cached`

2 Vision as plan B

The AI normally works from text only. If a text-only attempt fails, there is one retry with screenshots. Skipped if the quota is gone.

`use_vision`

3 Reconciliation

After the grader finds wrong fields in Supabase, this fixes them one at a time with the AI (prompt 3). It runs on its own thread, on its own ports.

`run_reconciliation_pass`

4 Smaller jobs

_emit sends progress events to the dashboard. get_store gives everyone the same cache connection. resolve_path_variant picks which version of the workflow a record follows.

`_emit · get_store · resolve_path_variant`

Grading never asks the agent

never reads RunResult.success

verify_batch · after the batch

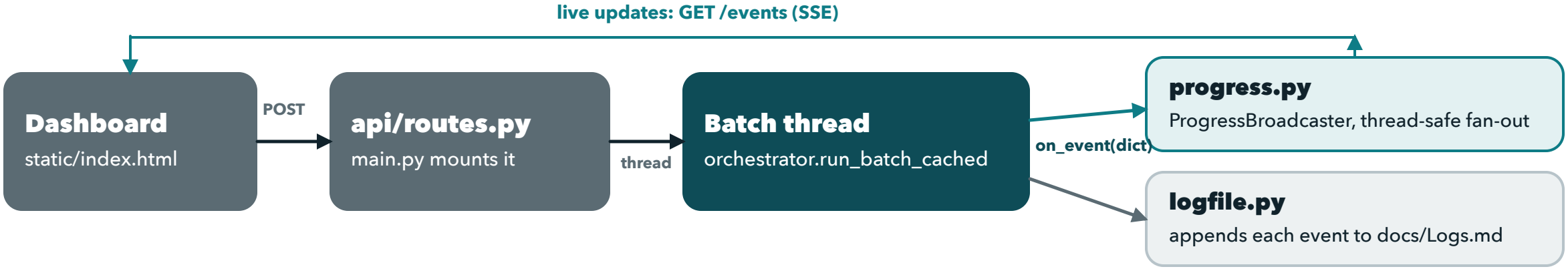
1. **Read the table once**
   One select-all into memory, not one query per record.
2. **Match each record by email**
   Case and whitespace insensitive; first matching row wins.
3. **Compare every field**
   values_match normalizes dates, phones, money and text first.
4. **Report**
   BatchReport: field accuracy, with 'row missing' kept apart from 'one field wrong'.

BatchReport

RECONCILIATION · own thread, after grading

1. **Build the to-do**
   Wrong fields on rows that did land. Skip empty values and open circuits.
2. **Fix via the edit flow**
   Live agent: search the person, open edit, change one field, save.
3. **3 at a time, own ports**
   Ports 9333 + 100 + n, so it can never clash with a running batch.
4. **Count and feed back**
   Fixed or skipped, feeding the circuit breaker and the event stream.

How a batch is driven and watched



`POST /run/start`
starts a batch on a background thread

`POST /run`
same batch, blocking (curl, compare.py)

`POST /run/abort`
skips unstarted records, never kills live ones

`GET /events`
Server-Sent Events: every progress event

`GET /status · /results`
progress, and the last saved BatchReport

`GET /cache/stats`
hit rate per workflow

orchestrator.py imports nothing from FastAPI. It only calls on_event(dict), so the same code runs from a script or a test.

A 100-record batch, in order

plumbing · LLM · replay · grading

1 2 3 4 5 6 7

Load and queue

100 records loaded and scheduled. 10 get a slot, 90 wait.

`loader.py`

`routes.py`

First record teaches

Wins the seed lock. The LLM fills the form while the recorder saves each step.

`runner.py`

`instrumentation.py`

`store.py`

The rest wait

Other slot holders sit at the seed lock. No browser is open yet.

`orchestrator.py`

Replay fans out

Cache populated: records replay 10 wide with Playwright, no LLM.

`engine.py`

Misses heal

A failed step gets one scoped LLM call, then the cache is corrected.

`prompts.py`

`runner.py`

Writes confirmed

Each submit waits for the database response. Windows close, slots free up.

`orchestrator.py`

Grade and clean up

Independent read of the table, BatchReport, then reconciliation on its own thread.

`evaluator.py`

`results_store.py`

Built, and what to decide next

Built

- ✓ Cold run teaches the cache; replay runs with no LLM
- ✓ Scoped fallback that self-heals, guarded by a circuit breaker
- ✓ 10 isolated Chrome windows, one CDP port each
- ✓ Independent grading, then reconciliation of wrong fields
- ✓ Live dashboard over SSE and a Logs.md transcript per run

Open edges • for discussion

- → A second, differently shaped target form, to prove portability (prompts and the write-confirmation URL are DataDesk/Supabase-specific)
- → Wizard or multi-page forms, and more action types
- → Reconciliation fixes aren't re-verified yet
- → Hardening: stale field-less click rows, no per-field lock when a cached step goes stale, one source of truth for headless

Decision for today: which real form comes next, and what gets hardened first?